

MODULE 4 — Deadlocks

Senior SWE Interview Track — Operating Systems

4.1 Coffman Conditions

1. Why Interviewers Ask This

The four conditions are the checklist every deadlock answer must be built on: to prevent deadlock you break exactly one, and interviewers expect you to say *which one* your fix breaks.

2. Core Concept

Deadlock: a set of threads/processes where each waits for a resource held by another in the set — none can ever proceed. All four **Coffman conditions** must hold simultaneously: 1. **Mutual exclusion** — resources are exclusively held. 2. **Hold and wait** — holders request more while holding. 3. **No preemption** — resources can't be forcibly taken. 4. **Circular wait** — a cycle $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_n \rightarrow T_1$ of “waits for”.

3. Internal Working

Model as a **Resource Allocation Graph (RAG)**: edges $T \rightarrow R$ (request) and $R \rightarrow T$ (assignment). With single-instance resources, deadlock $\Leftrightarrow$ cycle in the RAG. With multi-instance resources, a cycle is necessary but not sufficient (someone outside the cycle may release).

4. ASCII Diagram

```
Classic AB-BA deadlock:
T1: lock(A) ... lock(B)      T2: lock(B) ... lock(A)

      holds          requests
T1 -----> A      T1 ----> B
T2 -----> B      T2 ----> A

RAG cycle: T1 -> B -> T2 -> A -> T1  => DEADLOCK
All 4 conditions hold: exclusive locks, holding-while-waiting,
no forced release, circular wait.
```

5. Real Production Example

- DB transactions: TX1 updates rows (1,2), TX2 updates (2,1) → MySQL/Postgres detect and abort one (“Deadlock found when trying to get lock”).
- Java services: two subsystems each take `lockA` then `lockB` in opposite orders under rare code paths — fires once a month at peak.

6. Advantages

The framework converts “it’s stuck” into a diagnosable checklist and names the exact lever each strategy pulls.

7. Trade-offs

Conditions 1–3 are usually *desirable* properties (that’s why circular wait is the one we usually break).

8. Common Mistakes

- Listing 3 of 4 conditions; forgetting all four must hold **simultaneously**.
- Calling any hang a “deadlock” (could be livelock, starvation, or a lost wakeup).
- Thinking a RAG cycle always means deadlock (multi-instance caveat).

9. Performance Implications

Deadlocks freeze throughput to zero for the involved set and everything queued behind their locks; even deadlock-prone designs force conservative, coarse locking that costs parallelism.

10. Common Interview Questions

- “List the four Coffman conditions and give a two-lock example.”
- “Which condition does lock ordering break?” (circular wait)

11. Follow-up Questions

- “Deadlock vs livelock vs starvation?” “Draw the RAG for this scenario — is it deadlocked?”

12. Coding/Debugging Scenarios

- `jstack` prints “Found one Java-level deadlock” with both stacks → read the cycle, fix the lock order.
- Two goroutines blocked forever on channels each expecting the other to send — same cycle, no locks (communication deadlock).

13. Best Practices

For every fix you propose, name the broken condition; treat locks + any blocking wait (channels, thread-pool tasks waiting on pool tasks) as RAG resources.

14. Practice Questions

1. Map each of these fixes to the condition it breaks: lock ordering, trylock-with-backoff, lock-free structures, acquiring all locks upfront.
2. Thread pool of size 4; every task submits a subtask to the same pool and waits for it. Deadlock? Which “resource” cycles? (pool threads — thread-pool starvation deadlock)

4.2 Deadlock Prevention

1. Why Interviewers Ask This

“How do you *prevent* deadlocks in your service?” is the practical senior question — the expected headline answer is **global lock ordering**.

2. Core Concept

Prevention = design so one Coffman condition can never hold: - **Break mutual exclusion**: share (immutable data, RW read side) or go lock-free (CAS). - **Break hold-and-wait**: acquire *all* resources at once, or release everything before re-acquiring. - **Break no-preemption**: `trylock` with timeout — on failure release what you hold and retry (preempt yourself). - **Break circular wait**: impose a **total order** on locks; always acquire in ascending order. Cycle impossible.

3. Internal Working

Lock ordering proof: if every thread acquires locks in increasing rank, then a waiting thread always waits for a lock ranked *higher* than all it holds → any wait chain is strictly increasing → no cycle. Implementation: rank by address (e.g., `lock(min(a,b))`; `lock(max(a,b))` for transfers), by fixed hierarchy documented per subsystem, or enforced at runtime (lockdep, custom lock-rank asserts).

4. ASCII Diagram

```
transfer(from, to):
  first = min(from.id, to.id)
  second = max(from.id, to.id)
  lock(first); lock(second)
  ...
  unlock(second); unlock(first)
```

T1: transfer(A,B) -> lock A, lock B
T2: transfer(B,A) -> lock A, lock B (same order!)

Wait-chains only go "up" the order
=> no cycle => no deadlock

5. Real Production Example

- Linux kernel: documented lock ordering + **lockdep** (runtime validator that flags any ordering violation ever observed, even before a real deadlock).
- Bank-transfer double-entry code ordering account locks by ID — the canonical interview snippet.
- DB engines latch B-tree pages in a fixed direction (root → leaf, left → right) for the same reason.

6. Advantages

Deadlock becomes *impossible*, not just unlikely; zero runtime cost for lock ordering; simple to review.

7. Trade-offs

- All-at-once acquisition: poor utilization, must know needs upfront.
- Timeout/backoff: converts deadlock into retries — can livelock; spurious timeouts under load.
- Global order: hard across module boundaries/callbacks; ordering may force holding locks longer than needed.

8. Common Mistakes

- Proposing timeouts as “prevention” without a backoff/jitter story (→ livelock).
- Lock ordering by object identity that can change (use stable IDs/addresses).
- Forgetting that callbacks/virtual methods invoked under a lock can acquire arbitrary locks (order violated invisibly).

9. Performance Implications

Prevention is cheapest at design time. Coarser “grab everything” approaches serialize; ordering costs nothing at runtime — that’s why it’s the default answer.

10. Common Interview Questions

- “You have `transfer(a, b)` called concurrently with `transfer(b, a)` — fix it.” (order by ID; mention tie/self-transfer)
- “Which Coffman condition does each technique break?”

11. Follow-up Questions

- “How do you enforce lock order across a 2M-line codebase?” (lockdep-style runtime checking, lint, lock hierarchy docs)
- “What if you can’t order because locks are discovered dynamically?” (trylock+backoff, or restructure)

12. Coding/Debugging Scenarios

- Implement `transfer` with address-ordered `std::scoped_lock(m1, m2)` (which internally uses a deadlock-avoidance algorithm) — know that `std::lock` exists.
- Add a debug build assert: thread-local list of held lock ranks; acquiring a lower rank than held → abort with both stacks.

13. Best Practices

Document a lock hierarchy; never call unknown code (callbacks, listeners) while holding locks; prefer one lock per subsystem until profiling demands finer grain; use scoped multi-lock APIs.

14. Practice Questions

1. Write deadlock-free `transfer(from, to, amount)` handling `from == to`.
 2. Your fix uses trylock with 50 ms timeout; under peak load transfers start failing in bursts. Explain and improve (jittered backoff, ordering instead).
-

4.3 Deadlock Avoidance

1. Why Interviewers Ask This

Tests whether you know the prevention/avoidance/detection taxonomy precisely — avoidance = runtime decisions using *future knowledge* (max claims), i.e., the Banker's algorithm family.

2. Core Concept

Avoidance grants a resource request **only if the resulting state is safe** — a state from which *some* execution order lets every process finish with its declared **maximum claim**. Unsafe $\neq$ deadlocked; unsafe = no guarantee. The system dodges unsafe states so deadlock can never be reached.

3. Internal Working

Each process declares max resource needs upfront. On each request, the system simulates: “pretend to grant; can I still find a completion order for everyone?” If yes → grant; else → make the requester wait. The check is the Banker's safety algorithm (next section).

4. ASCII Diagram

```
State space:
[ SAFE states ] --grant--> [ SAFE ]      always allowed
[ SAFE ] --grant--> [ UNSAFE ]          REFUSED (wait instead)
[ UNSAFE ] --unlucky requests--> [ DEADLOCK ]
Avoidance keeps the system inside the SAFE region at all times.
```

5. Real Production Example

Rare in OS kernels (max claims unknown), but the *pattern* is everywhere: admission control — a cluster scheduler (Kubernetes, YARN) only places a pod if the node can satisfy its declared resource *requests*; DB connection admission that refuses work that could exhaust the pool mid-transaction; memory reservations before starting a batch job.

6. Advantages

No deadlock and no aborted work (vs detection/recovery); better utilization than acquire-all-upfront prevention.

7. Trade-offs

Requires accurate max-claim declarations (rarely available); per-request $O(m \cdot n^2)$ safety check; conservative — refuses grants that would in practice be fine; fixed process/resource population assumptions.

8. Common Mistakes

- Conflating avoidance with prevention (prevention = structural impossibility; avoidance = dynamic safe-state checks).
- Saying unsafe = deadlocked.

9. Performance Implications

Safety-check cost per allocation + reduced concurrency from conservative refusals; that's the price of never aborting.

10–11. Interview & Follow-ups

- “Prevention vs avoidance vs detection — one line each + example.”
- “Why don't real OSes use avoidance?” (unknown max claims, dynamic processes, check cost)

12. Coding/Debugging Scenario

Design admission control for a worker that needs up to **m** DB connections per job from a pool of **P**: only admit a job if remaining pool $\geq$ its declared max after reserving others' outstanding maxima — that *is* banker's thinking.

13. Best Practices

Use avoidance-style admission control at system boundaries (declared quotas/requests), not fine-grained locks.

14. Practice Questions

1. Give a state that is unsafe yet never deadlocks. Why did avoidance still refuse it?
2. Map Kubernetes resource requests/limits onto avoidance vocabulary.

4.4 Banker's Algorithm (High Level)

1. Why Interviewers Ask This

Occasionally asked to run one small example by hand; mostly they test that you understand **safe sequence** reasoning.

2. Core Concept

Named for a banker who never lends cash such that clients' *maximum* future needs can't all eventually be met. Data: $Available[m]$, $Max[n][m]$, $Allocation[n][m]$, $Need = Max - Allocation$.

3. Internal Working

Safety check: $Work = Available$; repeatedly find any process with $Need_i \leq Work$; pretend it finishes ($Work += Allocation_i$); mark finished. If all can finish $\rightarrow$ safe (that order is a **safe sequence**). **Request check:** if $request \leq Need_i$ and $\leq Available$, tentatively grant and run the safety check; commit if safe, else the process waits.

4. ASCII Diagram

```
Available = 3      Alloc  Max  Need
                   P0:   5    10   5
                   P1:   2     4   2
                   P2:   2     9   7
Safe? Work=3: P1 (2<=3) finishes -> Work=5
                P0 (5<=5) finishes -> Work=10
                P2 (7<=10) finishes -> SAFE, sequence P1,P0,P2
If instead P2 held 3 (Available=2): P1->Work=4; P0 needs 5 >4; P2 needs 6 >4
-> UNSAFE (grant would be refused)
```

5. Real Production Example

Directly: some embedded/RTOS and mainframe allocators. Spiritually: cloud capacity reservation systems and pool admission control ("never commit capacity you can't cover if everyone calls their max").

6. Advantages

Provably deadlock-free with no aborts; handles multi-instance resources (where RAG cycles aren't sufficient).

7. Trade-offs

Needs max claims upfront; $O(m \cdot n^2)$ per request; assumes fixed populations; conservative refusals.

8. Common Mistakes

- Arithmetic slips in hand simulation (always recompute $Need = Max - Alloc$ first).
- Declaring "unsafe" after trying only one candidate order — you must try all (any process with $Need \leq Work$ works; greedy is fine because granting only increases $Work$).

9. Performance Implications

Fine for coarse, infrequent allocations (jobs, pools); far too slow for per-lock granularity.

10–11. Interview & Follow-ups

- “Run the banker’s check on this 3×3 table.” “What exactly is a safe sequence?” “Why is it impractical for general OS use?”

12. Coding/Debugging Scenario

Implement `is_safe(available, alloc, max)` in 20 lines; property-test it against a brute-force order search.

13. Best Practices

Reserve banker-style logic for admission control with declared quotas; keep the checked resource coarse.

14. Practice Questions

1. 5 processes, 3 resource types (classic textbook table) — find a safe sequence.
2. Prove: granting a request that passes the safety check can never lead the checker itself to deadlock.

4.5 Deadlock Detection

1. Why Interviewers Ask This

This is what real systems (databases, JVM tooling) actually do — expect “how does MySQL/`jstack` find deadlocks?”

2. Core Concept

Let deadlocks happen; periodically (or on lock-wait) search for cycles in the **wait-for graph** (WFG: edge $T_i \rightarrow T_j$ if T_i waits for a resource T_j holds); on detection, break the cycle via recovery.

3. Internal Working

- Single-instance resources: DFS cycle detection in WFG, $O(V+E)$.
- Multi-instance: run a banker-like reduction with *current requests* (no Max needed): repeatedly “finish” any process whose request $\leq$ Work; unfinished set at fixpoint = deadlocked.
- Triggers: on each blocking wait (InnoDB checks when a lock wait starts), on a timer, or on demand (`jstack`, `SHOW ENGINE INNODB STATUS`).

4. ASCII Diagram

```
Wait-for graph:
TX1 --waits-for--> TX2 --waits-for--> TX3
  ^                               |
  +----- waits-for -----+
DFS finds cycle {TX1,TX2,TX3} -> pick victim (e.g., fewest undo records)
-> abort TX -> its locks release -> others proceed
```

5. Real Production Example

- **InnoDB**: detects on lock wait, rolls back the transaction with the smallest undo log, returns error 1213 — your app must retry.
- **Postgres**: checks the WFG after `deadlock_timeout` (default 1 s) of waiting.
- **JVM**: `jstack/JMX findDeadlockedThreads()` reports monitor cycles (but can’t fix them — process restart).
- Distributed systems mostly use **timeouts + retry** because building a global WFG is impractical.

6. Advantages

No upfront knowledge; zero overhead until contention; maximum concurrency in the common (non-deadlocked) case.

7. Trade-offs

Work is lost on abort; detection latency = stalled throughput window; requires recovery machinery; in-process deadlocks (mutexes) usually have no safe recovery except restart.

8. Common Mistakes

- Suggesting detection for plain in-process mutexes without noting you can't safely "abort" a thread holding locks (state corruption) — detection fits *transactional* systems where rollback exists.
- Forgetting the app-side retry after a DB deadlock error (a real prod bug: users see 1213).

9. Performance Implications

Detection cost scales with waiters (InnoDB deadlock checks can themselves become hot under extreme contention — it has a `innodb_deadlock_detect` off-switch that falls back to `innodb_lock_wait_timeout`).

10. Common Interview Questions

- "How does a database detect deadlocks?" "How would you detect a deadlock in a running JVM/Go service?" (jstack; Go: goroutine dump via SIGQUIT/pprof)

11. Follow-up Questions

- "How does the victim get chosen?" (least work to undo / fewest locks / lowest priority)
- "Why do distributed systems prefer lock timeouts over WFG detection?" (no global view, partial failure)

12. Coding/Debugging Scenarios

- Prod incident: order-service errors `1213 Deadlock found`. Steps: read `SHOW ENGINE INNODB STATUS` latest deadlock section → identify the two statements and index order → fix access order or add retry with jitter.
- Hung Java service: `jstack <pid>` → "Found one Java-level deadlock" → capture, restart, then fix ordering.

13. Best Practices

Always wrap transactional writes in idempotent retry; keep transactions short and touch rows in consistent order; alert on deadlock counters, not just errors.

14. Practice Questions

1. Implement WFG cycle detection given `waits_for: map<Tid, Tid>`.
2. Two UPDATE statements deadlock via different secondary indexes — explain how index choice creates opposite row-lock orders.

4.6 Deadlock Recovery

1. Why Interviewers Ask This

Completes the detection story: once found, *someone must lose*. Tests judgment about victim selection and starvation.

2. Core Concept

Options: 1. **Abort processes**: all in the cycle (brutal) or one at a time (re-detect after each) — victim selection by priority, work done, rollback cost, locks held, interactivity. 2. **Resource preemption**: forcibly take resources; requires the victim to **rollback** to a safe point; risk of the same victim losing repeatedly → **starvation** (bound it by counting rollbacks / aging).

3. Internal Working

Transactional systems make recovery clean: undo logs restore state, locks release atomically. Non-transactional (in-memory mutexes): no safe preemption — recovery = kill/restart the process (crash-only design), which is why prevention matters more inside a process.

4. ASCII Diagram

```
detect cycle {TX1,TX2,TX3}
  victim = argmin(rollback_cost) = TX2
  rollback TX2 -> release its locks
  TX1, TX3 proceed; TX2 retried (with backoff; count retries!)
  Repeated victimization of TX2 -> starvation -> raise its priority (aging)
```

5. Real Production Example

InnoDB rolls back the smallest transaction; Postgres aborts the waiter that triggered detection; Kubernetes-style systems “recover” by killing and rescheduling pods (preemption); microservices recover via request timeout + retry, treating the whole call as the abortable unit.

6. Advantages

Lets you run optimistically at full concurrency and pay only when deadlock actually happens (rare in well-ordered systems).

7. Trade-offs

Lost work, retry storms if uncoordinated (add jitter), starvation of repeat victims, complexity of choosing rollback points (checkpointing).

8. Common Mistakes

- No starvation bound on victim selection.
- Retrying immediately without jitter → the same collision replays (retry storm / livelock).
- Assuming you can kill one *thread* in a process safely.

9. Performance Implications

Recovery cost = wasted work × deadlock rate; if deadlock rate climbs, redesign locking rather than tuning recovery.

10–11. Interview & Follow-ups

- “You detected a deadlock among 3 transactions — who dies and why?”
- “How do you prevent the victim from starving?” (rollback counter → eventually never the victim)

12. Coding/Debugging Scenario

Implement DB-deadlock retry middleware: catch 1213/40P01, exponential backoff with jitter, max 3 attempts, idempotency guard.

13. Best Practices

Make units of work idempotent and retryable; short transactions; bounded, jittered retries; log every deadlock with both parties for offline fixing.

14. Practice Questions

1. Design victim selection for a payment system where TXs differ 1000× in size.
2. Why does “abort all in the cycle” ever make sense? (fast recovery when re-detection is expensive; cycles usually size 2 anyway)

4.7 Livelock

1. Why Interviewers Ask This

The favorite trap after you propose “trylock and retry”: states change, but no progress. Distinguishing deadlock/ livelock/starvation crisply is a senior marker.

2. Core Concept

Livelock: threads are *not blocked* — they actively run, respond to each other, and keep changing state — yet no one makes progress. Deadlock frozen; livelock busy.

3. Internal Working

Classic genesis: both threads take lock A/B respectively, fail trylock on the other, *both* release, *both* retry at the same cadence → perpetual collision. Same dynamics: two people side-stepping in a hallway; distributed leader election with synchronized retries; message redelivery ping-pong between two queues.

4. ASCII Diagram

```
T1: lock(A) tryB FAIL release(A)  lock(A) tryB FAIL release(A) ...
T2: lock(B) tryA FAIL release(B)  lock(B) tryA FAIL release(B) ...
    |_____ perfectly synchronized retries = livelock _____|
Fix: randomized jitter -> T1 retries at +7ms, T2 at +23ms -> one wins.
```

5. Real Production Example

- Retry storms: all clients time out together, retry together, overload the server, time out again (thundering-herd livelock) — fixed by exponential backoff **with jitter** (AWS's canonical guidance).
- CSMA/CD Ethernet used randomized backoff for exactly this.
- Two services each rolling back and retrying a distributed transaction on conflict, colliding forever.

6–7. Advantages / Trade-offs

(N/A — defect.) The relevant trade-off: retry-based deadlock fixes liveness risk; jitter + backoff + retry budget is the standard antidote.

8. Common Mistakes

- Defining livelock as “a kind of deadlock” — states differ (RUNNING vs BLOCKED), detection differs (livelock invisible to WFG/deadlock detectors!).
- Backoff without randomness (synchronized clients stay synchronized).

9. Performance Implications

Livelock burns 100% CPU/network while delivering zero goodput — often *worse* than deadlock operationally, and harder to spot because everything “looks busy”.

10–11. Interview & Follow-ups

- “Deadlock vs livelock vs starvation — definitions + one example each.”
- “Your trylock-retry fix — what new failure mode did you introduce and how do you mitigate it?” (livelock; jittered exponential backoff, retry budgets)

12. Coding/Debugging Scenario

CPU pegged, throughput ~0, no thread is blocked (all RUNNABLE in dumps), logs show rapid acquire/release cycles → livelock; add jitter and a progress metric alert.

13. Best Practices

Every retry loop: exponential backoff + full jitter + max attempts + circuit breaker; prefer ordering (prevents) over retry (avoids badly).

14. Practice Questions

1. Convert the AB-BA trylock livelock into a correct solution two ways (ordering; jittered backoff).
2. How would you *detect* livelock in production metrics? (high CPU + flat goodput + high retry counters)

4.8 Starvation (in the deadlock context)

1. Why Interviewers Ask This

Completes the liveness taxonomy. Asked as: “no deadlock, no livelock — can a thread still wait forever?” Yes: starvation.

2. Core Concept

Starvation: a thread is perpetually *able* to proceed but never chosen — others always win the resource. System-wide progress exists; individual progress doesn't. Causes: strict priorities, unfair locks (barging), reader-preferring RW locks vs writers, repeated deadlock-victim selection, greedy neighbors.

3. Internal Working

Unfair (barging) mutexes hand the lock to whoever CASes first on release — a just-arrived spinning thread beats a long-sleeping waiter (it's cache-hot and already running) → the sleeper can lose indefinitely. Fair (FIFO/ticket) locks fix this at ~2× handoff cost (must wake a specific sleeper instead of letting a runner barge). Java `ReentrantLock(true)`, Go mutex starvation mode (FIFO after 1 ms wait), ticket spinlocks are all “fairness switches”.

4. ASCII Diagram

```
Unfair lock timeline (T3 starves):
lock free -> T1 (barged) -> free -> T2 (barged) -> free -> T1 ...
                T3 asleep in wait queue: loses every race
Fair/FIFO lock: grant order = arrival order -> T3 bounded wait
Trade: fairness costs throughput (forced wakeups, cold caches)
```

5. Real Production Example

Go added mutex starvation mode (1.9) after real services saw tail-latency blowups from barging; writer starvation on reader-heavy RW locks delaying config updates; low-priority batch pods never scheduled in saturated clusters (fixed by priority aging / guaranteed quotas).

6–7. Advantages / Trade-offs

Unfairness is a deliberate *throughput optimization* (barging exploits cache-hot threads); fairness bounds tail latency at the cost of mean throughput. Say this trade explicitly — it's the senior insight.

8. Common Mistakes

- Confusing starvation with deadlock (starved thread proceeds instantly if granted; deadlocked never can).
- Assuming locks are fair by default (most aren't).
- Fixing starvation with fair locks everywhere (pay the cost only where tail latency matters).

9. Performance Implications

Symptom signature: healthy p50, monstrous p99.9 for a subset of threads/requests; overall throughput fine. Detect by per-thread wait-time histograms, not averages.

10–11. Interview & Follow-ups

- “Deadlock vs livelock vs starvation table.” “Why are unfair locks the default?” “How does Go's mutex starvation mode work?”

12. Coding/Debugging Scenario

One consumer thread of ten processes ~0 items under load; dumps show it always in lock wait → barging starvation; enable fair mode for that lock or shard the queue.

13. Best Practices

Bound waiting somewhere (fair mode after threshold — Go's design is the template); monitor max wait per waiter class; aging for priority systems (Module 2).

14. Practice Questions

1. Fill the 3×4 table: {deadlock, livelock, starvation} × {thread state, CPU usage, detectable by WFG?, canonical fix}.
2. Design a lock that is unfair for throughput but guarantees a 10 ms starvation bound.

Module 4 Cheat Sheet (one page)

Coffman (all four required): mutual exclusion · hold-and-wait · no preemption · circular wait.

Strategy	Idea	Breaks	Real example
Prevention	Make a condition impossible	Usually circular wait (lock ordering)	Order account locks by ID; kernel lockdep
Avoidance	Grant only if state stays safe	Dodges unsafe states	Banker's; K8s admission by declared requests
Detection	Find WFG cycles, then recover	—	InnoDB (abort smallest TX), Postgres 1 s check, jstack
Recovery	Abort victim / preempt+rollback	—	DB rollback + app retry w/ jitter
Ignore	"Ostrich" — restart on hang	—	Most in-process mutex code, honestly

Liveness taxonomy:

	Thread state	CPU	WFG-detectable	Fix
Deadlock	Blocked forever	0%	Yes	Ordering / detection+abort
Livelock	Running, no progress	High	No	Jittered backoff
Starvation	Ready, never chosen	others'	No	Fairness / aging

Golden answers: fix AB-BA by total lock order (min/max ID); DB deadlock → automatic victim rollback → app must retry with jittered backoff; trylock-retry introduces livelock → add jitter; unfair locks trade tail latency for throughput.

Top Interview Questions

1. Four Coffman conditions + two-thread example.
2. Fix `transfer(a,b)` vs `transfer(b,a)` — write the ordered-locking code.
3. Prevention vs avoidance vs detection — one-liners + where each is used.
4. Run Banker's safety check on a small table.
5. How does MySQL/Postgres handle deadlocks end-to-end (detection → victim → app retry)?
6. Deadlock vs livelock vs starvation.
7. Thread-pool deadlock: tasks waiting on subtasks in the same pool.

Common Mistakes (module-wide)

- Missing "all four simultaneously"; calling every hang a deadlock.
- Trylock-retry without jitter (livelock); retries without idempotency.
- Believing DB handles deadlocks "fully" — the app retry is your job.
- Proposing thread-kill recovery for in-process mutexes.

- No starvation bound in victim selection or priority schemes.

Mock Interview (self-test, ~20 min)

1. (Code) Write deadlock-free `transfer` including self-transfer and same-ID edge cases; state the invariant that proves no cycle.
2. (Scenario) Payments deadlock ~50×/day in InnoDB between two known queries. Walk from `SHOW ENGINE INNODB STATUS` to the code fix.
3. (Depth) Your colleague replaces all locks with `tryLock(100ms)` + retry. List every new failure mode and your mitigation.
4. (Design) A workflow engine runs user-defined steps that lock arbitrary entities. Design its anti-deadlock strategy (ordering by entity ID? detection? timeouts?) and defend it.
5. (Trap) `jstack` shows no monitor cycle, but the service is frozen and CPU is 95%. What class of failure do you hunt, and how? (livelock)